

Python进阶语法-判断, 循环与函数

1. 判断

1.1. 逻辑运算符

逻辑运算符用于组合条件。常见的逻辑运算符有 `and`、`or` 和 `not`

1.1.1. `and`

`and` 运算符可理解为数学上的逻辑 "并"

▼ `and`运算符示例

Python |

```
1 x = 10
2 if x > 5 and x < 15:
3     print("x is between 5 and 15")
```

1.1.2. `or`

`or` 运算符可理解为数学上的逻辑 "或"

▼ `or`运算符示例

Python |

```
1 x = 3
2 if x < 5 or x > 10:
3     print("x is either less than 5 or greater than 10")
```

1.1.3. `not`

`not` 运算符可理解为数学上的逻辑 "非"

▼ `not`运算符示例

Python |

```
1 x = 5
2 if not x > 10:
3     print("x is not greater than 10")
```

1.2. 运算符优先级

运算符优先级决定了在表达式中哪个运算符先执行，了解运算符优先级有助于编写正确的判断语句

优先级	运算符
强制优先级	加括号 <code>()</code>
1	算术运算符 <code>+, -, *, /, //, %, **</code>
2	比较运算符 <code><, <=, >, >=, ==, !=</code>
3	逻辑运算符 <code>not, and, or</code>

▼ 运算符优先级代码示例

Python |

```
1 x = 5
2 y = 10
3 z = 15
4
5 # 运算符优先级示例
6 if (x + y) * z > 100:
7     print("The result is greater than 100")
```

2. 循环

2.1. while 循环

`while` 循环用于在满足某个条件的情况下重复执行一个代码块。`while` 循环首先检查条件表达式的值。如果条件为 `True`，则执行代码块。执行完代码块后，重新检查条件表达式。如果条件仍为 `True`，再次执行代码块。这一过程会一直重复，直到条件为 `False`，语法如下

▼ while循环伪代码

Python |

```
1 while 条件:
2     代码块
```

while循环示例

Python |

```
1 x = 0
2 while x < 5:
3     print(x)
4     x += 1
```

注意点：

- `while` 的条件语句需要是布尔类型。
- 使用 `while` 循环务必检查，避免无限循环，除非真的需要。

`while`可以嵌套使用

while循环嵌套示例

Python |

```
1 x = 0
2 while x < 3:
3     y = 0
4     while y < 2:
5         print(f"x={x}, y={y}")
6         y += 1
7     x += 1
```

2.2. for 循环

`for` 循环用于遍历序列（如列表、元组、字符串等）或任何可迭代对象。`for` 循环从可迭代对象中获取第一个元素，将其赋值给循环变量，执行 `for` 循环体中的代码，然后获取下一个元素，直到遍历完可迭代对象中的所有元素，语法如下：

for循环伪代码

Python |

```
1 for 临时变量 in 可迭代对象:
2     代码块
```

for循环示例

Python |

```
1 for i in [1, 2, 3, 4, 5]:
2     print(i)
```

2.2.1. range 函数

`range` 函数用于生成一个数字序列，通常用于 `for` 循环中，语法如下

▼ range()函数伪代码

Python |

```
1 range(start, stop, step)
```

- `start` : 序列的起始值，默认值为 0
- `stop` : 序列的终止值（不包括该值）
- `step` : 序列中数字之间的差值，默认值为 1

▼ range()函数示例

Python |

```
1 ▼ for i in range(5): # 生成 0 到 4 的序列
2     print(i)
3
4 ▼ for i in range(1, 10, 2): # 生成 1 到 9 的序列，步长为 2
5     print(i)
```

2.2.2. for 循环可以嵌套使用

▼ for循环嵌套示例

Python |

```
1 ▼ for i in range(3):
2     for j in range(2):
3         print(f"i={i}, j={j}")
```

2.2.3. for 中的临时变量

临时变量只作用于循环内部，但实际上循环外部也可以访问到（不符合规范）。尽量避免在外部访问，可能导致错误（检查不出来的那种）

▼ for循环示例

Python |

```
1 ▼ for number in range(3):
2     print(number)
3     print(number) # 虽然可以访问到，但不推荐这样做
4
5     # 如果接下来你没有重新赋值number变量，而是粗心直接使用，代码结果会出错，并且难以检查出来
6 ▼ if number < 5:
7     print(number)
```

2.3. break 和 continue

- **break** : 用于提前退出循环（已达到目的 / 有错误）。
- **continue** : 用于跳过某次循环，进行下一次。

while中break和continue示例

Python |

```
1  # while 使用 break 提前退出循环
2  while i < 5:
3      print("hello")
4      break
5  # 上面代码只会输出一次hello
6
7  # while 使用 continue 跳过某次循环
8  while i < 5:
9      if i < 4:
10         continue
11     print("hello")
12 # 上面代码只会输出一次hello
```

for中break和continue示例

Python |

```
1  # for 使用 break 提前退出循环
2  for i in range(10):
3      if i == 5:
4          break
5      print(i) # 输出 0 到 4
6
7  # for 使用 continue 跳过某次循环
8  for i in range(10):
9      if i % 2 == 0:
10         continue
11     print(i) # 输出奇数 1, 3, 5, 7, 9
```

注意作用域: **break** 和 **continue** 仅作用在所在的循环上

```
1 # 下面的例子中，break 语句仅作用在内层循环上，只退出内层循环，不影响外层循环的执行
2 for i in range(3): # 外层循环
3     for j in range(5): # 内层循环
4         if j == 3:
5             break # 仅退出内层循环
6         print(f"i={i}, j={j}")
7     ''
8 输出如下
9 i=0, j=0
10 i=0, j=1
11 i=0, j=2
12 i=1, j=0
13 i=1, j=1
14 i=1, j=2
15 i=2, j=0
16 i=2, j=1
17 i=2, j=2
18 ''
```

3. 函数

在 Python 中，函数是一种代码块，用于执行特定的任务。函数通过 `def` 关键字定义，支持参数传递和返回值。函数的使用使代码更加模块化和可重用

3.1. 函数的定义方法

函数的定义语法如下：

```
1 def 函数名(参数列表):
2     代码块
```

```
1 def greet(name):
2     print(f"Hello, {name}!")
3
4 # 调用函数
5 greet("Alice") # 输出内容: Hello, Alice
```

在上面的示例中，我们定义了一个名为 `greet` 的函数，该函数接受一个参数 `name` 并打印问候语

3.2. 函数的参数

3.2.1. 形参与实参

形式参数（形参）： 是定义函数时使用的参数名称

实际参数（实参）： 是调用函数时传递给函数的具体值

```
1 def add(a, b): # a 和 b 是形参
2     return a + b
3
4 result = add(3, 5) # 3 和 5 是实参
5 print(result) # 输出: 8
```

3.2.2. 函数的传参方式

3.2.2.1. 位置传参

函数调用时，可以不指定具体的参数名称，函数自动根据顺序一个个对应，按照参数定义的顺序传递参数，这种方式称为位置传参

```
1 def describe_pet(animal_type, pet_name):
2     print(f"I have a {animal_type} named {pet_name}.")
3
4 describe_pet("hamster", "Harry") # 输出I have a hamster named Harry.
```

3.2.2.2. 关键词传参

调用函数时可以通过参数名称来传递参数，这样可以不按照顺序传参，这种方式尤其适用于函数有多个参数时，防止参数传递的混淆

▼ 函数关键词传参示例

Python |

```
1 def describe_pet(animal_type, pet_name):
2     print(f"I have a {animal_type} named {pet_name}.")
3
4     describe_pet(pet_name="Whiskers", animal_type="cat")
```

3.2.2.3. 函数指定默认参数

函数可以为参数设置默认值，调用函数时可以省略这些参数

▼ 函数默认参数示例

Python |

```
1 def greet(name, message="Hello"):
2     print(f"{message}, {name}!")
3
4     greet("Alice") # 输出: Hello, Alice!
5     greet("Bob", "Hi") # 输出: Hi, Bob!
```

3.2.3. 函数的返回值

函数可以使用 return 语句返回一个值，如果没有 return 语句，函数默认返回 None。

▼ 函数返回值示例

Python |

```
1 def add(a, b):
2     return a + b
3
4     result = add(3, 5)
5     print(result) # 输出: 8
6
7 def no_return():
8     print("This function has no return statement")
9
10    result = no_return()
11    print(result) # 输出: None
```


函数的多返回值：函数可以返回多个值，这些值会被打包成一个元组。调用函数时，可以使用多重赋值（解包）来分别获取这些返回值

函数返回值示例

Python |

```
1 def get_coordinates():
2     x = 10
3     y = 20
4     return x, y
5
6 x_coord, y_coord = get_coordinates()
7 print(f"x: {x_coord}, y: {y_coord}") # 输出: x: 10, y: 20
```

3.3. 函数作为参数传递

在 Python 中，函数可以作为参数传递给另一个函数。这使得高阶函数和回调函数的使用成为可能

函数作为参数传递实例

Python |

```
1 def add(a, b):
2     return a + b
3
4 def subtract(a, b):
5     return a - b
6
7 def apply_operation(a, b, operation):
8     return operation(a, b)
9
10 result = apply_operation(5, 3, add)
11 print(result) # 输出: 8
12
13 result = apply_operation(5, 3, subtract)
14 print(result) # 输出: 2
```

3.4. 函数嵌套使用

函数可以在另一个函数内部定义，称为嵌套函数或内部函数。嵌套函数可以访问其外部函数的变量和参数

```
1 def outer_function(text):
2     def inner_function():
3         print(text)
4
5     inner_function()
6
7 outer_function("Hello from outer function!") # 输出: Hello from outer func
tion!
```

3.5. 变量的作用域

3.5.1. 局部变量

在函数内部定义的变量，只能在该函数内部访问。它们的生命周期仅限于函数的执行期间

```
1 def local_scope():
2     x = 5 # 局部变量
3     print(x) # 输出: 5
4
5 local_scope()
6 # print(x) # 会报错, 因为 x 是局部变量, 函数外不可访问
```

3.5.2. 全局变量

在函数外部定义的变量，可以在整个脚本中访问。函数内部可以使用 `global` 关键字对其进行修改

```
1 x = 10 # 全局变量
2
3 def modify_global():
4     global x
5     x = 5 # 修改全局变量
6
7 modify_global()
8 print(x) # 输出: 5
```

3.6. 函数的注释规范

以下是一些流行常用的函数注释规范，实际情况下，可根据自己的喜好（更多的是根据团队的规范）进行注释

函数的注释规范1

Python |

```
1 def add(a, b):
2     """
3     计算两个数的和
4
5     参数:
6     a (int, float): 第一个加数
7     b (int, float): 第二个加数
8
9     返回:
10    int, float: 两个数的和
11    """
12    return a + b
```

函数的注释规范2

Python |

```
1 def add(a, b):
2     """
3     计算两个数的和
4     :params a (int, float): 第一个加数
5     :params b (int, float): 第二个加数
6     :return sum(int, float): 两个数的和
7     """
8     return a + b
```